

Digital Assignment - I

class: C1 + TCI

i) Predictive parser String $\Rightarrow (a, (a, a))$

$$S \rightarrow a | \uparrow | (T)$$

$$T \rightarrow T, S | S$$

Solⁿ :

Step-1 : Elimination of left recursion.

$$T \rightarrow T, S | S$$

$$\begin{aligned} T &\rightarrow ST' \\ T' &\rightarrow , ST' | \epsilon \end{aligned}$$

Step-2 : Rules to compute first & follow of Non terminals.

$$S \rightarrow a | \uparrow | (T)$$

$$\text{FIRST}(S) = \{a, \uparrow, (\}$$

$$T \rightarrow ST'$$

$$\text{FIRST}(T) = \text{FIRST}(S) = \{a, \uparrow, (\}$$

$$T' \rightarrow , ST' | \epsilon$$

$$\text{FIRST}(T') = \{ , , \epsilon \}$$

$$\text{Follow}(S) = \{ \$ \}$$

$$T' \rightarrow , ST' \\ \alpha \quad B \quad \beta$$

$$\text{Follow}(S) = \text{First}(T') - \epsilon$$

$$= \{ , , \epsilon \} - \epsilon = \{ , \}$$

$$T' \rightarrow , ST' \\ \alpha \quad B \quad \beta$$

$\text{First}(T') \Rightarrow \text{contains } \epsilon$

$$\text{Follow}(S) = \text{Follow}(T')$$

$$S \rightarrow (T) \\ \alpha \quad B \quad \beta$$

$$\text{Follow}(T) = \text{First}() = \{) \}$$

$$T \rightarrow ST' \\ \alpha B$$

$$\text{Follow}(T') = \text{Follow}(T) = \{ \} \}$$

$$T \rightarrow ST' \\ \alpha B \beta$$

$$\text{Follow}(S) = \text{Follow}(T) = \{ \} \}$$

$$\text{first}(T') \Rightarrow \epsilon$$

NT	FIRST	FOLLOW
S	{a, ↑, (}	{\$, ,,)}
T	{a, ↑, (}	{) }
T'	{ ,, ε}	{) }

Step-3 = predictive parsing table.

NT	Input Symbol					
	a	↑	(	)	,	\$
S	$S \rightarrow a$	$S \rightarrow \uparrow$	$S \rightarrow ($	Error	Error	Error
T	$T \rightarrow ST'$	$T \rightarrow ST'$	$T \rightarrow ST'$			
T'				$T' \rightarrow \epsilon$	$T' \rightarrow ,ST'$	

$$M(S, a) = S \rightarrow a$$

$$M(T, a) = T \rightarrow ST'$$

$$M(S, \uparrow) = S \rightarrow \uparrow$$

$$M(T, \uparrow) = T \rightarrow ST'$$

$$M(S, () = S \rightarrow ($$

$$M(T, () = T \rightarrow ST'$$

$$M(T', ,) = T' \rightarrow ,ST'$$

$$M(T',)) = T' \rightarrow \epsilon$$

Step-4 ÷ Parse the String

②

STACK	INPUT	OUTPUT
S \$	(a(a,a)) \$	
(T) \$	(a(a,a)) \$	$S \rightarrow (T)$
T) \$	a(a,a)) \$	
ST') \$	a(a,a)) \$	$T \rightarrow ST'$
aT') \$	a(a,a)) \$	$S \rightarrow a$
T') \$	,(a,a)) \$	(a
,ST') \$	, (a,a)) \$	$T' \rightarrow ,ST'$
ST') \$	(a,a)) \$	(a,
(T) T') \$	(a,a)) \$	$S \rightarrow (T)$
T) T') \$	a,a)) \$	(a,(
ST') T') \$	a,a)) \$	$T \rightarrow ST'$
aT') T') \$	a,a)) \$	$S \rightarrow a$
T') T') \$	,a)) \$	(a,(a
,ST') T') \$	,a)) \$	$T' \rightarrow ,ST'$
ST') T') \$	a)) \$	(a,(a,
aT') T') \$	a)) \$	$S \rightarrow a$

$T') T') \$$	$)) \$$	$(a, (a, a$
$\epsilon) T') \$$	$)) \$$	$T' \rightarrow \epsilon$
$) T') \$$	$)) \$$	
$T') \$$	$) \$$	$(a, (a, a)$
$\epsilon) \$$	$) \$$	$T' \rightarrow \epsilon$
$) \$$	$) \$$	
$\$$	$\$$	$(a, (a, a)$

String parsed ✓

2) check whether the following is LL(1) grammar

$$S \rightarrow iEtS \mid iEtSeS \mid a$$

$$E \rightarrow b$$

Solⁿ ÷

Step-1 ÷ Left factoring

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

$$E \rightarrow b$$

To check if it is LL(1) we need to construct

Predictive parsing table, if there are multiple entries ⁽³⁾ then it is not LL(1) Grammar.

Step-2 : compute FIRST(), FOLLOW().

$$\text{First}(S) = \{i, a\}$$

$$\text{First}(S') = \{e, \epsilon\}$$

$$\text{First}(E) = \{b\}$$

$$\text{Follow}(S) = \{\$ \} \quad (\text{Rule-1})$$

$$S' \rightarrow \underset{\alpha}{e} \underset{\beta}{S} \quad \text{Follow}(S) = \text{Follow}(S') \quad (\text{Rule-2})$$

$$S \rightarrow \underset{\alpha}{iE} \underset{\beta}{+SS'} \quad \text{Follow}(S') = \text{Follow}(S) \quad (\text{Rule-2})$$

$$S \rightarrow \underset{\alpha}{i} \underset{\beta}{E} \underset{\beta}{+SS'} \quad \text{Follow}(E) = \text{first}(+SS') - \epsilon \quad (\text{Rule-2})$$

$$= \{+\}$$

$$S \rightarrow \underset{\alpha}{iE} \underset{\beta}{+} \underset{\beta}{S} \underset{\beta}{S'} \quad \text{Follow}(S) = \text{first}(S') - \epsilon \quad (\text{Rule-2})$$

$$= \{e\}$$

NT	first	follow
S	{i, a}	{\$, e}
S'	{e, \epsilon}	{\$, e}
E	{b}	{+}

Step-3 : construct predictive parsing table.

NT	Input Symbol						
	i	+	a	e	b	\$	
S	$S \rightarrow iEtSS'$	error	$S \rightarrow a$	error	error	error	{i, a}
S'	error	error	error	$S' \rightarrow es$ $S' \rightarrow \epsilon$	error	$S' \rightarrow \epsilon$	{e, e} { \$, e }
E	error	error	error	error	$E \rightarrow b$	error	{b}

$$M(S, i) = S \rightarrow iEtSS'$$

$$M(S, a) = S \rightarrow a$$

$$M(E, b) = E \rightarrow b$$

$$M(S', e) = S' \rightarrow es$$

$$M(S', \$) = S' \rightarrow \epsilon$$

$$M(S', e) = S' \rightarrow \epsilon$$

$$M(S', e) \Rightarrow \left. \begin{array}{l} S' \rightarrow es \\ S' \rightarrow \epsilon \end{array} \right\} \text{Multiple Entries}$$

"Not LL(1) Grammar"

3) construct operator precedence & parse id=id (4)

$$S \rightarrow L = R \mid R$$

$$L \rightarrow * R \mid id$$

$$R \rightarrow L$$

Solⁿ :

NT	Leading	trailing
S	{=, *, id}	{=, *, id}
L	{*, id}	{*, id}
R	{*, id}	{*, id}

operator precedence table

	=	*	id	\$
=		<	<	>
*	>	>	<	>
id	>	>		>
\$	<	<	<	

$a < b$ op. NT

$= R \quad = < \{*, id\}$

$* R \quad * < \{*, id\}$

$a > b$ NT. op

$L = \{*, id\} > =$

$\$ < \{=, *, id\}$

$\{=, *, id\} > \$$

Parse the String

id = id

\$ id = id \$

\$ < id > = < id > \$

$L \rightarrow id$

\$ L = < id > \$

$L \rightarrow id$

\$ L = L \$

Remove NT

\$ = \$

Relations

\$ < = > \$

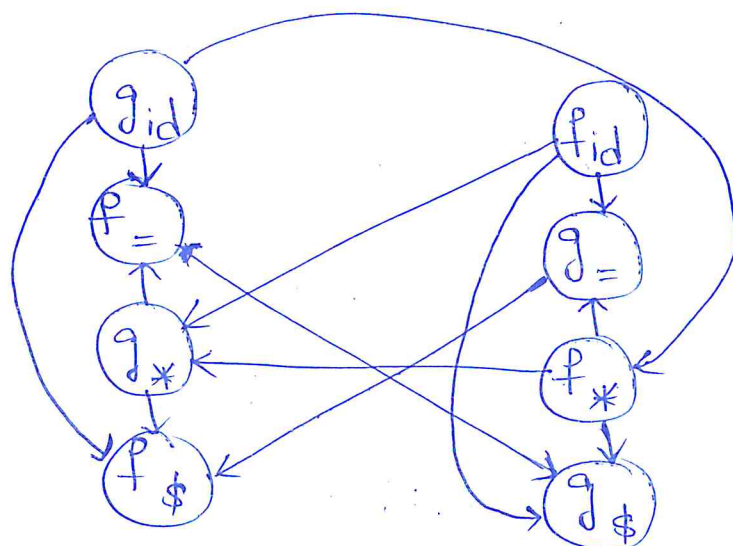
$S \rightarrow L = R$

\$ \$

Parsed

String parsed ✓

operator precedence function



$f_ = < g_*$

$f_ = < g_{id}$

$f_ = > g_ \$$

No cycle

table :

	*	=	id	\$
f	2	1	3	0
g	2	1	3	0

4) Recursive descent parsing. & check $id * id + id \$$ ⑤

$E \rightarrow T + E \mid T$

$T \rightarrow V * T \mid V$

$V \rightarrow id$

Solⁿ :

Procedure E {

 T();

 if (lookahead = '+') {

 Match('+');

 E();

 }

 if (lookahead = '\$') {

 Print("Success");

 } else {

 Error();

 }

}

Procedure T {

 V();

 if (lookahead = '*') {

 Match('*');

 T();

 }

Procedure V {

 if (lookahead = 'id') {

 Match('id');

 } else {

 Error();

 }

}

Procedure Match (token t) {

 if (lookahead = 't') {

 lookahead = next_token;

 } Else {

 error();

 }

Procedure Error () {

 Print ("Error");

}

String $\rightarrow$ id * id + id \$

it will enter E() then T() then V()

it matches 'id'

it gets back to T() & matches '*'

it enters T() & then V() & matches 'id'

it comes back E() & matches '+'

it enters E() again & then T() then V()

it matches 'id' & reaches '\$'

"Success"